

# Python for Artificial Intelligence

## Day 5: Dictionaries & Sets

*Month 1: Python Mastery | LLM & Agentic AI Track*

# Today's Agenda

- Hour 1: Key-value pairs, creating and accessing dictionaries, .get(), .items()
- Hour 2: Modifying dictionaries; set operations (union, intersection, difference)
- 15-Minute Break
- Hour 3: Weekly Skill Checkpoint, all of Week 1's core data structures

# Day 4 Quick Review (10 mins)

*Before we start, try these on your own:*

1. Create a list of 5 numbers and slice out the middle three.

2. What is the difference between a list and a tuple?

3. Build a tuple of two values, then unpack them into two separate variables.

4. Reverse a list without using reverse() or slicing.

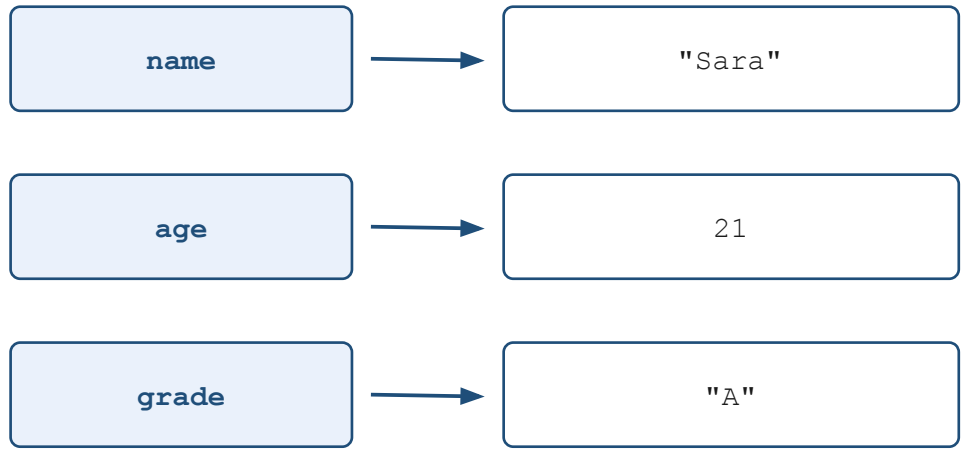
5. Why can't a list be used as a dictionary key, but a tuple can?



# Hour 1: Dictionaries

# Dictionaries: Key-Value Pairs

- A dictionary stores data as key-value pairs, written with curly braces
- Keys must be unique and immutable (strings, numbers, tuples)
- Look up a value instantly by its key, no searching required
- Written as { key: value, key: value }




*key → value, instant lookup*

# dict: Full Syntax

**SYNTAX**

```
my_dict = {key1: value1, key2: value2}
empty_dict = {}
```

**PARAMETERS**

**`key: value, ...`**: Any number of comma-separated pairs. Keys must be unique and immutable (strings, numbers, or tuples).

**YOU CAN**

✓ Use strings, numbers, or tuples as keys

✓ Store any type as a value, even another list or dictionary

✓ Create an empty dictionary with {} and add keys to it later

**YOU CANNOT**

✗ Use a list as a key, lists are mutable, and keys must be hashable

✗ Have two identical keys, the second one silently overwrites the first

✗ Assume keys are sorted, they appear in the order you inserted them, not alphabetical or numeric order

***Returns:*** *N/A, this is a literal, not a function call.*

# Dictionary Access, [] vs .get(): Full Syntax

**SYNTAX**

```
my_dict[key]
my_dict.get(key)
my_dict.get(key, default)
```

**PARAMETERS**

**`key`**: The key to look up.

**`default`**: Optional, only for .get(). The value to return if the key is missing. Defaults to None.

**YOU CAN**

- ✓ Use [] when you're certain the key exists
- ✓ Use .get(key) to safely check for a key without crashing
- ✓ Use .get(key, default) to supply your own fallback value

**YOU CANNOT**

- ✗ Use [] on a missing key and expect None back, it raises KeyError instead
- ✗ Pass a default value to [], square brackets don't accept one

***Returns:*** *The value for that key. .get() returns None (or your default) instead of raising an error.*

# Creating & Accessing a Dictionary

Access a value with square brackets or the safer .get() method.

```
student = {"name": "Sara", "age": 21, "grade": "A"}

print(student["name"])
print(student.get("age"))
print(student.get("email", "not found"))
```

**QUESTION**

# Think & Practice

Given inventory = {"pens": 50, "pencils": 30}, print the number of pencils using [], then safely check for "erasers" using .get() with a default of 0.

**ANSWER**

# Answer

```
inventory = {"pens": 50, "pencils": 30}
print(inventory["pencils"])
print(inventory.get("erasers", 0))
```

# Why .get() Is Safer Than []

[] raises a KeyError on a missing key. .get() just returns None, or your default.

```
student = {"name": "Sara", "age": 21}

print(student.get("grade"))          # None, no error
print(student.get("grade", "N/A"))   # N/A, your own default
# print(student["grade"])            # this would raise KeyError
```

**QUESTION**

# Think & Practice

Given settings = {"volume": 70}, safely check for a "brightness" setting, defaulting to 50 if it isn't set, and print the result.

ANSWER

# Answer

```
settings = {"volume": 70}
brightness = settings.get("brightness", 50)
print(brightness)
```

# .items(): Full Syntax

**SYNTAX**

```
for key, value in my_dict.items():
    ...
```

**YOU CAN**

✓ Unpack each pair directly into two loop variables, key and value

✓ Loop over just .keys() or just .values() instead, if you only need one side

✓ Convert the result to a list of tuples with list(my_dict.items())

**YOU CANNOT**

✗ Add or remove keys from the dictionary while looping over .items(), this raises a RuntimeError

✗ Call .items() on a list or a set, it's a dictionary-only method

***Returns:*** *A view of (key, value) tuples, one per pair in the dictionary.*

# Looping Through a Dictionary

.items() gives you both the key and the value together.

```
prices = {"apple": 120, "banana": 60, "mango": 200}

for item, price in prices.items():
    print(f"{item}: Rs. {price}")
```

EXAMPLE 1

# Practice: Build a Dictionary from Input

Type this code yourself and run it.

```
contact = {}
contact["name"] = input("Name: ")
contact["phone"] = input("Phone: ")

print(contact)
print(f"Saved contact for {contact['name']}")
```

**QUESTION 1**

# Think & Practice

Given a dictionary of student names and marks, print the name of the student with the highest marks.

**ANSWER**

# Answer 1

```
marks = {"Ali": 78, "Sara": 92, "Ahmed": 85}
best_student = None
best_score = -1

for name, score in marks.items():
    if score > best_score:
        best_score = score
        best_student = name

print(best_student)
```



# Hour 2: Modifying Dictionaries & Sets

# Modifying a Dictionary: Full Syntax

**SYNTAX**

```
my_dict[key] = value      # add or update
del my_dict[key]          # remove
```

**PARAMETERS**

**`key`**: The key to add, update, or remove.

**`value`**: The value to associate with that key.

**YOU CAN**

✓ Use the same line to add a brand-new key or update an existing one, Python figures out which

✓ Assign any type as the new value, even replacing a number with a string

✓ Remove a key you're sure exists with del

**YOU CANNOT**

✗ Use del on a key that doesn't exist, this raises KeyError, same risk as square-bracket access

✗ Undo a del, once removed, the key-value pair is gone

***Returns:*** *N/A, these are statements, not function calls that return a value.*

# Adding, Updating & Removing

Dictionaries are mutable, just like lists.

```
student = {"name": "Sara", "age": 21}

student["grade"] = "A"      # add
student["age"] = 22         # update
del student["grade"]        # remove

print(student)
```

**QUESTION**

# Think & Practice

Given profile = {"username": "kat22"}, add an "email" key, then update "username" to a new value. Print the final dictionary.

ANSWER

# Answer

```
profile = {"username": "kat22"}
profile["email"] = "kat22@email.com"
profile["username"] = "kat_23"
print(profile)
```

# set: Full Syntax

**SYNTAX**

```
my_set = {1, 2, 3}
empty_set = set()   # NOT {}, that creates an empty dict
```

**PARAMETERS**

`item1, item2, ...`: Any number of comma-separated values. Duplicates are automatically dropped.

**YOU CAN**

✓ Build a set from a list to remove duplicates: set([1, 2, 2, 3])

✓ Mix types, as long as every item is itself immutable

✓ Check membership fast with in, sets are optimized for this

**YOU CANNOT**

✗ Write {} expecting an empty set, that always creates an empty dictionary, use set() instead

✗ Put a list inside a set, lists are unhashable, this raises TypeError

✗ Rely on any particular order, sets are unordered, the printed order may not match insertion order

***Returns:*** *N/A, this is a literal, not a function call.*

# add(): Full Syntax

**SYNTAX**

```
my_set.add(item)
```

**PARAMETERS**

**`item`**: The value to add. Must be immutable (a number, string, or tuple).

**YOU CAN**

✓ Add a new, unique item

✓ Call add() with a value already in the set, nothing happens, no error, no duplicate

**YOU CANNOT**

✗ Add a list or another set directly, both are unhashable, this raises TypeError

✗ Add more than one item per call, only one value per add()

***Returns:*** *None. The set is modified in place.*

# discard(): Full Syntax

**SYNTAX**

```
my_set.discard(item)
```

**PARAMETERS**

`item`: The value to remove, if present.

**YOU CAN**

✓ Call it on a value that isn't in the set, safely, nothing happens

✓ Use it as your default choice for removing from a set, when you're not sure the value exists

**YOU CANNOT**

✗ Expect an error when the value is missing, that's what makes discard() different from list's remove()

✗ Get back the removed value, discard() always returns None

***Returns:*** *None. The set is modified in place, whether or not the item existed.*

# Sets: Unique, Unordered Values

Written with curly braces, duplicates are removed automatically.

```
fruits = {"apple", "banana", "apple", "cherry"}
print(fruits)          # duplicates gone
print(len(fruits))     # 3

fruits.add("mango")
fruits.discard("banana")
```

QUESTION

# Think & Practice

Given tags = {"python", "ai"}, add "ml" to it, then try adding "python" again. Print the set both times to see what changes.

ANSWER

# Answer

```
tags = {"python", "ai"}
tags.add("ml")
print(tags)

tags.add("python")
print(tags)   # unchanged, "python" was already there
```

# Set Operators, | & -: Full Syntax

**SYNTAX**

```
A | B      # union
A & B      # intersection
A - B      # difference
```

**PARAMETERS**

`A, B`: Two sets to combine or compare.

**YOU CAN**

✓ Combine these with parentheses for more complex comparisons: (A | B) - C

✓ Use them on sets built from lists, e.g. set(list1) & set(list2)

✓ Chain multiple sets: A | B | C

**YOU CANNOT**

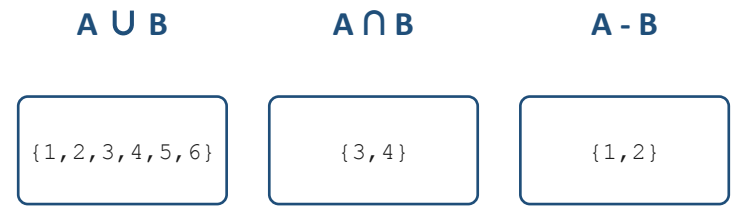
✗ Use these operators directly on lists, convert to a set first, list1 & list2 raises TypeError

✗ Expect A - B to equal B - A, difference is not symmetric, order matters

***Returns:*** *A new set. Neither original set is changed.*

# Set Operations

Union, intersection, and difference, common interview topics.




```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B)   # union
print(A & B)   # intersection
print(A - B)   # difference
```

**QUESTION**

# Think & Practice

Given A = {1, 2, 3, 4, 5} and B = {4, 5, 6, 7}, print everything in A that is not in B.

**ANSWER**

# Answer

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7}
print(A - B)
```

EXAMPLE 1

# Practice: Word Frequency Counter

Type this code yourself and run it.

```
text = "the quick brown fox jumps over the lazy dog the fox runs"
words = text.split()

counts = {}
for word in words:
    counts[word] = counts.get(word, 0) + 1

print(counts)
```

**QUESTION**

# Think & Practice

Given results = ["pass", "fail", "pass", "pass", "fail"], use a dictionary and .get() to count how many "pass" and how many "fail" results there are.

ANSWER

# Answer

```
results = ["pass", "fail", "pass", "pass", "fail"]
counts = {}

for result in results:
    counts[result] = counts.get(result, 0) + 1

print(counts)
```

EXAMPLE 2, INTERVIEW FAVORITE

# Practice: Find Common Items in Two Lists

Type this code yourself and run it.

```
list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]

common = set(list1) & set(list2)
print(common)
```

# zip(): Full Syntax

**SYNTAX**

```
zip(list1, list2)
```

**PARAMETERS**

`list1, list2, ...`: Two or more sequences to pair up position by position.

**YOU CAN**

✓ Pass more than two lists at once, zip(a, b, c) pairs all three

✓ Feed the result straight into dict() to build key-value pairs instantly

✓ Wrap it in list() to see the actual pairs while debugging: list(zip(a, b))

**YOU CANNOT**

✗ Expect it to pad a shorter list, zip stops as soon as the shortest input runs out

✗ Reuse the result twice, zip() produces a one-time-use sequence, wrap it in list() first if you need it again

***Returns:*** *A zip object of paired items. Usually wrapped in list() or dict() immediately.*

QUESTION 1, INTERVIEW STYLE

# Think & Practice (Harder)

Given two lists of the same length, build a dictionary that pairs each item from the first list with the item at the same position in the second list.

**ANSWER**

# Answer 1

```
names = ["Ali", "Sara", "Ahmed"]
marks = [78, 92, 85]

result = dict(zip(names, marks))
print(result)
```



# 15-Minute Break

Stretch, grab a coffee. We resume with the Week 1 Skill Checkpoint.



# Hour 3: Weekly Skill Checkpoint

# Weekly Skill Checkpoint

*"I can confidently manipulate all of Python's core data structures." Covers Days 1 to 5, work through this on your own.*

1. Write code that checks whether a given word is a palindrome, using string slicing, and prints True or False.

2. Given a list of numbers, build a new list containing only the ones that are both even and greater than 10.

3. Given a dictionary of product names and prices, find and print the cheapest product.

4. Given two lists of numbers, use sets to find which numbers appear in both.

5. Write a program that reads a sentence and prints a dictionary showing how many times each word appears.

6. Debug this on sight: a KeyError is raised when looking up a product that doesn't exist in a prices dictionary. Fix it using .get().

# Homework: Practice for Interviews

*Try these on your own before next class:*

1. Merge two dictionaries into one; if a key exists in both, keep the value from the second dictionary.
2. Write code that counts vowels and consonants in a sentence, using a dictionary to store the tally.
3. Given a list of email addresses, use a set to report how many are unique.
4. Write a program that groups a list of names by their first letter, into a dictionary of lists.
5. Explain, in your own words, when you would reach for a set instead of a list.

# Key Takeaways

- Dictionaries give instant key-based lookup; sets guarantee unique, unordered values
- .get() is almost always safer than [] for dictionaries you didn't build yourself
- Set operations (union, intersection, difference) solve "compare two collections" problems in one line
- This closes Week 1: control flow, loops, and all four core data structures are now yours

# Coming Up: Week 2

- Functions & Functional Thinking
- Defining functions, parameters, default and keyword arguments
- *args, **kwargs, and local vs. global scope
- Lambda functions, comprehensions, map(), filter(), and higher-order functions